

NetPlan - Spezifikation zum Nachbau

Stand: 28.8.2026

Diese Beschreibung ist als **Bauanleitung** gedacht, nicht als Beschreibung des Bestehenden. Sie nennt jede Festlegung, die getroffen wurde, und begründet die, die nicht offensichtlich sind. Die Rechenformeln stehen vollständig in [berechnungslogik.md](#) und werden hier nur referenziert.

Inhalt

1. [Zweck und Abgrenzung](#)
2. [Rahmenbedingungen](#)
3. [Technologieentscheidungen](#)
4. [Architektur](#)
5. [Datenmodell](#)
6. [Import-Pipeline](#)
7. [Rechenkern](#)
8. [Oberfläche](#)
9. [Persistenz](#)
- .0. [Export](#)
- .1. [Zweisprachigkeit](#)
- .2. [Erscheinungsbild](#)
- .3. [Karte](#)
- .4. [Designsystem](#)
- .5. [Auslieferung als Einzeldatei](#)
- .6. [Teststrategie](#)
- .7. [Gemessene Grenzen](#)
- .8. [Fallstricke](#)
- .9. [Empfohlene Baureihenfolge](#)

1. Zweck und Abgrenzung

Fachliche Frage: Welche Produktkategorie sollte in welchem Distributionszentrum gelagert werden?

Was das Werkzeug tut: Es liest historische Versanddaten und einen extern erstellten Forecast ein, ermittelt daraus den Bedarf je Kategorie und Kundenregion, leitet über eine Zielreichweite den Ziel-Bestand ab und bewertet jeden Standort anhand dreier gewichteter Kriterien. Ergebnis ist eine begründete Zuordnungsempfehlung, wahlweise als Alleinzuordnung oder als Aufteilung auf mehrere Standorte.

Was es bewusst nicht tut: keine mathematische Optimierung, keine Routenplanung, keine eigene Prognose, keine Bestandsoptimierung nach Servicegrad, keine Mehrstufigkeit. Diese Abgrenzung ist wichtig, weil sie den Umfang klein und das Ergebnis erklärbar hält.

2. Rahmenbedingungen

Diese Vorgaben haben praktisch jede spätere Entscheidung bestimmt:

VORGABE	KONSEQUENZ
Läuft per Doppelklick aus <code>file://</code>	Keine ES-Module (CORS), kein Build-Schritt, keine relativen <code>fetch</code> -Aufrufe
Kein Server, keine Installation, keine Adminrechte	Alles clientseitig, Bibliotheken lokal eingebunden
Corporate-Netz, CDNs oft gesperrt	Keine externen Ressourcen zur Laufzeit; alles vendored
Daten verlassen den Rechner nicht	Kein Netzverkehr außer optionalen Kartenkacheln
Weitergabe per Mail/Netzlaufwerk	Zusätzliche Einzeldatei-Fassung mit allem inline
Zielgruppe Desktop, gemischte Browser	Konservative Syntax, keine exotischen APIs

3. Technologieentscheidungen

Sprache: JavaScript im klassischen ES5-Stil — kein `const / let`, keine Pfeilfunktionen, keine Template-Literale, keine Klassen. Verwendet werden nur wenige neuere Standardfunktionen (`Object.assign`, `Array.find`, `Array.findIndex`, `String.padStart`), die jeder aktuelle Browser kann. Dazu HTML und CSS. Kein TypeScript, kein Framework, kein Präprozessor.

Begründung: Jeder Übersetzungsschritt hätte die Kernanforderung gebrochen. ES-Module scheitern unter `file://` an der CORS-Sperre; klassische `<script>`-Tags nicht.

Fremdbibliotheken (alle lokal, keine CDN-Aufrufe):

BIBLIOTHEK	VERSION	ZWECK	GRÖSSE
PapaParse	5.4.1	CSV-Parser mit Trennzeichenerkennung	19 kB
SheetJS <code>xlsx.core.min.js</code>	0.18.5	Excel lesen und schreiben	427 kB
Chart.js (UMD)	4.4.4	Diagramme	201 kB
Leaflet	1.9.4	Karte	144 kB + CSS

Wichtig: die **core**-Variante von SheetJS, nicht `full`. Letztere enthält binäre Codepage-Tabellen und ist keine gültige UTF-8-Datei — sie lässt sich nicht in eine HTML-Datei einbetten.

Entwicklungswerkzeuge (nicht Teil der Auslieferung): Node.js für die Build-Skripte, Playwright für die Tests.

4. Architektur

Ein globales Namensobjekt `window.LNP`, in das sich Module per IIFE eintragen. Geladen wird über klassische `<script>`-Tags in fester Reihenfolge.

```
window.LNP = {  
  il8n, util, geo,      Werkzeuge, ohne Abhängigkeit untereinander  
  state,                Datenhaltung + Ereignisse  
  sim,                  Rechenkern, ohne DOM-Zugriff  
  charts, mapview,      Darstellungsschicht  
  dcs, data, simui,     Ansichten  
  dashboard, scenarios,  
  exporter, demo, app   Navigation und Start  
}
```

Vier tragende Muster:

Ein Zustand, Änderungen per Ereignis. `state` hält ein einziges Objekt. Wer etwas ändert, ruft `emit('dcs')`; Module haben sich mit `onChange(fn)` registriert und zeichnen sich selbst neu. Kein Modul greift in ein anderes hinein. Das ist der Grund, warum Sprachwechsel, Szenarienwechsel und Projekt-Laden die gesamte Oberfläche konsistent aktualisieren.

Ereignisnamen: `dcs`, `records`, `regions`, `settings`, `assignments`, `scenarios`, `project` (alles neu), `reset`.

Rechenkern getrennt von der Oberfläche. `sim.js` kennt kein DOM, keine Farben, keine Texte. Nur Zahlen rein, Zahlen raus. Das erlaubt, den Kern für Messreihen und Tests direkt anzusprechen.

Rendern über HTML-Strings. Render-Funktionen bauen HTML zusammen und setzen es per `innerHTML`. Klicks laufen über Event-Delegation am Container (ein Listener je Tabelle, nicht je Zeile). Einfach, schnell genug, kein Framework.

Fallbacks als Grundprinzip. Kartenkacheln blockiert → Schema-Ansicht; Leaflet fehlt → SVG; Speicher voll → Konfiguration retten, Rohdaten verwerfen, Hinweis zeigen; Übersetzung fehlt → Originaltext. Nichts bricht ab.

Umfang zur Orientierung: rund 5.500 Zeilen JavaScript in 15 Modulen, 730 Zeilen HTML, 800 Zeilen CSS.

5. Datenmodell

5.1 Distributionszentrum

FELD	TYP	BEDEUTUNG	
id	String	intern erzeugt	
name	String	Pflicht	
code	String	Kurzcode für Diagrammachsen	
region, country	String	auch Grundlage der Koordinatensuche	
lat, lng	Number \	null	null = unbekannt
capacity	Number	Stellplätze	
usedSlots	Number	Grundbelegung durch nicht simulierte Sortimente	
storageCostPerSlotMonth	Number \	null	null = Netzwerk-Standardwert
handlingCostPerPallet	Number \	null	null = Standardwert
transportBasePerPallet	Number \	null	null = Standardwert
transportCostPerKm	Number \	null	null = Standardwert
fixedCostPerPeriod	Number	fließt nur in Netzwerk-KPIs	
active	Boolean	inaktive DCs sind von der Simulation ausgenommen	
regionCosts	{ regionKey: EUR }	Pauschale je Palette, überschreibt die Distanzrechnung	

Die Unterscheidung **null gegen 0** ist wesentlich: null bedeutet „nicht gesetzt, nimm den Netzwerkwert“, 0 bedeutet „kostet nichts“.

5.2 Datensatz (Historie und Forecast, gleiches Schema)

FELD	TYP	BEMERKUNG			
dataset	`history` \ `forecast`	einzigster Unterschied zwischen beiden			
customer, country, region	String	frei			
regionKey	String	`region`	country	customer	'Ohne Region'
category	String	Pflicht			
period	String	normalisiert, z.B. 2027-03			
periodTs	Number	Zeitstempel zum Sortieren			
periodDays	Number	echte Länge der Periode in Tagen			
qty, revenue, volume, pallets, palletEq	Number	Kennzahlen			
lat, lng	Number \ NaN	optional aus der Datei			

regionKey ist der Schlüssel, über den Bedarf, Koordinaten und Transportpauschalen zusammenfinden. Er muss beim Import einmal festgelegt und danach überall konsistent verwendet werden.

5.3 Region

{ name, country, lat, lng, source } mit source aus datei | automatisch | manuell | offen. Die Quelle sichtbar zu machen ist wichtig — der Nutzer muss erkennen, welche Koordinate geraten ist.

5.4 Zuordnung (je Kategorie)

```
{ mode: 'single'|'split'|'manual', targetDays, createdAt,
  dataset, periodFrom, periodTo,
  parts: [ { dcId, dcName, share, pallets, slots,
            transportCost, storageCost, handlingCost, totalCost,
            avgDistance, avgTransitDays, score, feasible, scores{...} } ],
  regions: [ { regionKey, dcId, pallets, distance, cost } ],
  metrics: { ... } }
```

`parts[].slots` ist die kapazitätswirksame Größe: Sie belegt Platz im DC und beeinflusst die Bewertung aller später zugeordneten Kategorien. `regions` wird für Karte und Export mitgeführt.

5.5 Szenario

Vollständige Kopie von `settings`, `dc`s und `assignments` plus ein KPI-Schnappschuss. **Nicht** kopiert werden die importierten Daten — Szenarien vergleichen Konfigurationen, nicht Datenstände.

5.6 Einstellungen mit Standardwerten

SCHLÜSSEL	STANDARD	BEDEUTUNG
qtyPerPallet	500	Stück je Palette
volPerPallet	1.8	m³ je Palette
stockFactor	1.0	Sicherheitsaufschlag auf den Ziel-Bestand
horizonDaysOverride	null	null = aus den Daten ableiten
targetDaysGlobal	21	Zielreichweite in Tagen
targetDaysByCategory	{}	überschreibt den globalen Wert
costPerPalletKm	0.045	EUR
costBasePerPallet	6	EUR
storageCostPerSlotMonth	12	EUR
handlingCostPerPallet	3.5	EUR
kmPerDay	500	für die Transitzeit
handlingDays	0.5	fixer Anteil der Transitzeit
weights	{capacity:30, transport:45, service:25}	Reglerwerte, werden normiert
maxUtilization	0.85	Ziel-Auslastungsgrenze
autosave	true	localStorage
mapMode	'auto'	Kartenmodus
branding	siehe Abschnitt 12	Logo und Bezeichnungen

6. Import-Pipeline

6.1 Dateien lesen

.xlsx .xlsm .xlsb .xls über SheetJS (cellDates: true , Blattauswahl bei mehreren Blättern), alles andere über PapaParse mit header: true und automatischer Trennzeichenerkennung. Erste Zeile = Spaltenüberschriften.

6.2 Spaltenzuordnung

Frei konfigurierbar, mit Vorbelegung durch einen Bewertungsalgorithmus. Je Feld existiert eine Synonymliste (deutsch und englisch). Bewertung jeder Kombination aus Feld und Spaltenüberschrift:

BEDINGUNG	PUNKTE
exakte Übereinstimmung	100
Übereinstimmung ohne Leerzeichen	95
Überschrift enthält Synonym	60 + Länge des Synonyms
Synonym enthält Überschrift (ab 3 Zeichen)	40 + Länge der Überschrift

Alle Paare werden gesammelt, **absteigend nach Punktzahl vergeben**, und jede Spalte wird höchstens einmal verwendet. Der letzte Punkt ist wichtig: Bei feldweiser Zuordnung kann eine Spalte wie „Transportkosten je km“ gleichzeitig als Grundkosten und als Kilometersatz gelesen werden.

Pflichtfelder: Kategorie, Periode, mindestens eine Kennzahl, mindestens ein Ortsbezug. Fehlt eines, bleibt der Import gesperrt.

6.3 Periodenerkennung

Normalisiert auf einen Schlüssel plus Zeitstempel plus echte Tageszahl:

EINGABE	ERGEBNIS	TAGE
2027-03 , 2027-03-15	2027-03	Monatslänge
15.03.2027 , 03/2027	2027-03	Monatslänge
Mär 27 , Mar 2027	2027-03	Monatslänge
2027-Q2	2027-Q2	91,3
2027-KW12	2027-KW12	7
2027	2027	365
Excel-Seriennummer 20000-60000	Monat	Monatslänge
unbekannt	Text unverändert	30,44

Zweistellige Jahreszahlen werden zu 20xx ergänzt. Blanke Jahreszahlen gelten nur zwischen 1900 und 2200.

6.4 Zahlenerkennung

Muss deutsche und englische Formate in derselben Datei verkraften:

1. Einheiten entfernen (EUR € \$ % Stk m³ kg), Leerzeichen entfernen
2. Punkt **und** Komma vorhanden: das zuletzt auftretende Zeichen ist das Dezimaltrennzeichen
3. nur Komma: Tausendertrennung genau dann, wenn das Muster `^-?[1-9]\d{0,2}(\,\d{3})+$` passt, sonst Dezimalkomma
4. nur Punkt: spiegelbildlich dieselbe Regel

Die führende [1-9] ist kein Detail: Ohne sie wird 0,042 als 42 gelesen.

6.5 Verdichtung

Bei mehr als 2.000 Zeilen werden Datensätze auf die Dimensionskombination `dataset | customer | country | region | category | period` verdichtet, Kennzahlen summiert. Das senkt den Speicherbedarf ohne Genauigkeitsverlust, weil die Simulation ohnehin auf dieser Ebene rechnet.

6.6 Koordinaten

Eingebaute Nachschlagetabelle mit rund 200 Schreibweisen (Länder, deutsche Bundesländer, grobe Vertriebsgebiete wie „Nord“/„Süd“/„Benelux“, größere Logistikstandorte). Zerlegt zusammengesetzte Bezeichnungen an `/ | , > -` und versucht Teiltreffer ab vier Zeichen. Jede Koordinate ist manuell überschreibbar; die Herkunft wird angezeigt.

7. Rechenkern

Vollständig in `berechnungslogik.md`. Die Kurzfassung:

```
Paletten    = PalÄq → Paletten → Volumen ÷ VolProPal → Menge ÷ StückProPal
Tage        = Summe der echten Tageslängen aller Perioden im Filter
Bedarf/Tag  = Paletten ÷ Tage
ZielBestand= Bedarf/Tag × Zielreichweite × Sicherheitsaufschlag
Distanz     = Haversine × 1,28
€/Palette   = Regionspauschale, sonst Grundkosten + €/km × Distanz
Transit     = Handlingtage + Distanz ÷ km pro Tag
```

Drei Teil-Scores je 0-100:

```
Kapazität:  Auslastung ≤ Grenze → 100 - 50 × (Auslastung ÷ Grenze)
            Grenze...100 %      → 50 × (1 - (Auslastung - Grenze) ÷ (1 - Grenze))
            darüber             → 0
Transport:  100 × günstigste €/Palette ÷ eigene €/Palette
Reichweite: 100 × (0,7 × Bestandsfähigkeit + 0,3 × Reaktionsfähigkeit)
Gesamt:     Σ normierte Gewichte × Teil-Scores
```

Schnittstelle des Kerns:

```
sim.demandFor({category, dataset, periodFrom, periodTo}) → Bedarf je Region
sim.evaluateDC(dc, allocation, ctx)                     → Kennzahlen eines DC
sim.runSingle(params) / runSplit(params) / runManual(params, shares)
sim.runAll(params)                                       → alle Kategorien
sim.applyResult(result)                                  → als Zuordnung übernehmen
```

Zwei Festlegungen, die man bewusst treffen muss:

- **Transport-Score ist relativ zum Bewerberfeld.** Der günstigste Standort erhält immer 100. Nimmt man ein DC aus dem Netz, steigen alle übrigen Werte. Die Alternative wäre eine absolute Skala mit willkürlichem Nullpunkt.
- **Fixkosten fließen nicht in den Standort-Score,** nur in die Netzwerk-Kennzahlen. Sonst trüge die erste zugeordnete Kategorie die gesamten Fixkosten eines Standorts.

Das Splitting arbeitet greedy: Kandidatenpool aus den besten n Standorten, Regionen absteigend nach Volumen, je Region das bestbewertete DC mit freier Kapazität, Rest anteilig weiter. Heuristisch, nicht optimal — das ist eine bewusste Entscheidung zugunsten von Nachvollziehbarkeit.

8. Oberfläche

Seitenleiste mit sechs Ansichten, Kopfzeile mit Sprache, Theme und Demodaten.

ANSICHT	INHALT
Dashboard	6–7 KPI-Kacheln, drei Diagramme, Karte, Detailtabelle
Daten & Import	Dropzone, Spalten-Mapping, Mengenlogik, Zielreichweite je Kategorie, Regionen mit Koordinaten, Datenbestand
DC-Verwaltung	Tabelle mit Anlegen/Bearbeiten/Duplizieren/Löschen, Modal mit allen Feldern und Regionspauschalen, Datei-Import
Simulation	Parameterspalte (Kategorie, Datenbasis, Periode, Modus, Zielreichweite, drei Gewichtungsregler, Auslastungsgrenze) und Ergebnisspalte
Szenarien	Speichern, Kennzahlenvergleich inklusive „aktueller Stand“, zwei Diagramme, Zuordnungsmatrix
Export & Projekt	Excel, CSV, PDF, Projektdatei, Erscheinungsbild, Kostenparameter, Formelübersicht

Die Ergebnisdarstellung der Simulation ist der Kern des Ganzen und sollte in dieser Reihenfolge aufgebaut sein:

1. **Empfehlungskarte**, hervorgehoben: Standort bzw. Aufteilung, ein Satz Begründung („Ausschlaggebend ist ... mit x von y Punkten“), Warnungen, acht Kennzahlen
2. **Rangliste aller Standorte** mit Gesamt-Score und den drei Teil-Scores
3. **Score-Zusammensetzung** als gestapelter Balken der gewichteten Beiträge
4. **Aufteilungstabelle** mit überschreibbaren Anteilen
5. **Regionale Zuordnung**: welche Region wird woher beliefert

Punkt 2 und 3 sind der Grund, warum das Werkzeug als Entscheidungshilfe taugt und nicht als Black Box: Der Nutzer sieht, welches Kriterium die Empfehlung getragen hat und wie knapp sie ausfällt.

9. Persistenz

localStorage, entprellt auf 400 ms, unter einem Schlüssel als JSON.

Bei Überschreiten der Quote (rund 5 MB, entspricht 15.000–18.000 Datensätzen): Rohdaten verwerfen, Konfiguration, Zuordnungen und Szenarien weiter speichern, Hinweis anzeigen. Der Arbeitsstand geht dadurch nicht verloren, muss aber über die Projektdatei gesichert werden.

Projektdatei als JSON mit allem: DCs, Datensätzen, Regionen, Zuordnungen, Szenarien, Einstellungen inklusive Logo. Das ist gleichzeitig das Format zur Weitergabe.

Jeder `localStorage`-Zugriff gehört in `try/catch` — unter `file://` und in restriktiven Browsereinstellungen wirft schon das Lesen.

10. Export

Excel über SheetJS, neun Blätter: KPI, DCs, Zuordnungen, Detailergebnisse, Regionszuordnung, Zielreichweiten, Regionen, Szenarien, Rohdaten (gedeckt auf 50.000 Zeilen). Spaltenbreiten setzen, Anteile als Prozentwerte schreiben.

CSV mit Semikolon, BOM voran, Dezimalkomma — damit Excel die Datei ohne Importdialog korrekt öffnet.

PDF ohne zusätzliche Bibliothek: eine Druckansicht im Dokument, per `@media print` sichtbar geschaltet, alles andere ausgeblendet, dann `window.print()`. Diagramme werden über `canvas.toDataURL()` als Bild übernommen. Der Nutzer wählt im Druckdialog „Als PDF speichern“.

Alle Downloads über `Blob` und `URL.createObjectURL`.

11. Zweisprachigkeit

Schlüssel des Wörterbuchs ist der **deutsche Originaltext**. Fehlt ein Eintrag, erscheint der deutsche Text — die Oberfläche bleibt immer funktionsfähig.

Zwei Wege, bewusst getrennt:

- **Statische Texte aus dem HTML** werden beim Start einmalig erfasst (Textknoten sowie `placeholder`, `title`, `aria-label`), mit normalisierten Leerräumen als Schlüssel und dem Original als Rückfallwert. Beim Sprachwechsel werden sie neu gesetzt. Der Zeitpunkt ist entscheidend: vor dem ersten Rendern, wenn im Dokument garantiert keine Nutzerdaten stehen.
- **Dynamische Texte** laufen über `t(text)` und `tf(text, ...)` mit `{0}`-Platzhaltern.

Der Aufwand lässt sich stark senken, indem die Übersetzung zentral in den Render-Helfern sitzt: Tabellenüberschriften, KPI-Beschriftungen, Legenden, Diagrammreihen, Meldungen und

Rückfragen. Dann bleiben nur zusammengesetzte Texte einzeln zu behandeln.

Ebenfalls umzustellen: Zahlenformat (`Intl.NumberFormat` mit `de-DE` bzw. `en-GB`), Datumsausgabe und die Spaltenüberschriften der Exporte. Importierte Daten werden **nicht** übersetzt.

Zur Kontrolle eignet sich ein automatischer Test, der die englische Oberfläche über alle Ansichten nach deutschen Wörtern und Umlauten durchsucht.

12. Erscheinungsbild

`branding: { appName, appSubtitle, initials, logo }`. Das Logo wird als `data:-URI` gespeichert, damit es in der Projektdatei mitreist. Obergrenze 300 kB, sonst frisst es den `localStorage`. Ohne Logo zeigt die Marke das Kürzel. Der Anwendungsname erscheint zusätzlich in der Titelzeile des Browsers, im PDF-Bericht und in der Excel-Auswertung.

Beim Sprachwechsel gilt: Der **Standard**-Untertitel wird übersetzt, ein selbst gesetzter bleibt unangetastet.

13. Karte

Drei Betriebsarten mit automatischem Abstieg:

1. **Kacheln** von OpenStreetMap über Leaflet
2. **Schema** — eine eigene `L.GridLayer`-Ableitung, die statt Kacheln ein Koordinatenraster auf Canvas zeichnet. Pan, Zoom und alle Marker funktionieren unverändert; die Anwendung bleibt vollständig offline nutzbar.
3. **SVG** — reine Streudarstellung, falls Leaflet gar nicht geladen ist

Der Wechsel von 1 nach 2 erfolgt selbsttätig nach drei `tileerror`-Ereignissen innerhalb von sechs Sekunden. Zusätzlich manuell wählbar, Auswahl wird gespeichert.

Dargestellt werden DCs (Radius nach Kapazität), Kundenregionen (Radius nach Volumen) und die Zuordnungslinien DC → Region, eingefärbt nach DC und in der Stärke nach Menge, gedeckelt auf die 400 stärksten.

Wer keine Außenverbindung will: Leaflets voreingestellten Verweis auf die eigene Website über `map.attributionControl.setPrefix('')` entfernen. Der Quellenhinweis der Kacheln muss als Text bleiben.

14. Designsystem

CSS-Variablen für Flächen, Text, Linien, acht Serienfarben und vier Statusfarben. Helles Schema auf `:root`, dunkles auf `:root[data-theme="dark"]`. Das Theme wird **vor** dem ersten

Zeichnen per Inline-Skript im `<head>` gesetzt, sonst blitzt das helle Schema auf.

Regeln für die Diagramme, die sich bewährt haben:

- Kategoriale Farben in **fester Reihenfolge**, nie zyklisch. Ab dem neunten Eintrag auf „Weitere“ verdichten statt Farben zu wiederholen.
- Identität nie allein über Farbe: ab zwei Serien immer eine Legende.
- Nur **eine** Werteachse. Zwei Größen unterschiedlicher Skala gehören in zwei Diagramme.
- Statusfarben (gut/Warnung/kritisch) sind reserviert und nie „Serie 4“.
- Zurückhaltendes Raster, 2 px Fläche zwischen gestapelten Segmenten, abgerundete Balkenenden.

Zahlen in Tabellen rechtsbündig mit `font-variant-numeric: tabular-nums`.

15. Auslieferung als Einzeldatei

Ein Build-Skript ersetzt `<link>` und `<script src>` durch Inline-Inhalte und bettet die von der CSS referenzierten Bilder als `data:-URI` ein. Ergebnis: eine HTML-Datei von rund 1 MB ohne jeden externen Verweis, die sich per Mail verschicken lässt.

Zwei Punkte, die man leicht übersieht: Vorkommen von `</script` im eingebetteten Code müssen maskiert werden, und alle eingebetteten Dateien müssen gültiges UTF-8 sein.

Die Entwicklung findet weiter in den Einzeldateien statt; die Einzeldatei ist nur die Auslieferungsform.

16. Teststrategie

Automatisierte Durchläufe im echten Browser mit Playwright, gegen die Datei unter `file://` — genau so, wie der Nutzer sie öffnet. Fünf Suiten:

1. **Hauptablauf:** Demodaten, DC-Verwaltung, Simulation einzeln und als Split, manuelle Anteile, alle Kategorien, Dashboard, Szenarien, Exporte, Theme, Neuladen mit Wiederherstellung
2. **Import:** deutsche CSV und englische XLSX mit abweichenden Layouts, Prüfung der automatischen Spaltenzuordnung und des Zahlenparsings
3. **Vorlagen:** die mitgelieferten Importvorlagen müssen ohne manuelles Mapping durchlaufen
4. **Sprache:** englische Oberfläche über alle Ansichten nach deutschen Resten durchsuchen
5. **Erscheinungsbild und PDF:** Logo-Zyklus, Druckansicht, Diagrammaufnahme

Dazu Messläufe, die den Rechenkern direkt ansprechen — für Grenzwerte und für die Zahlen in der Dokumentation.

Zwei Prüfungen haben besonders viel gefunden: Kartenkacheln im Test blockieren (deckt den Offline-Pfad ab) und Konsolenfehler als Testfehler werten.

17. Gemessene Grenzen

DATENSÄTZE	SIMULATION	ALLE KATEGORIEN	PROJEKTGRÖSSE	LOCALSTORAGE
5.000	2 ms	1,2 s	1,4 MB	funktioniert
25.000	6 ms	0,8 s	6,7 MB	Limit überschritten
100.000	44 ms	1,3 s	26,7 MB	Limit überschritten
250.000	109 ms	2,6 s	66,7 MB	Limit überschritten

Die Rechenzeit ist unkritisch; die Grenze ist der Browserspeicher. Zeithorizont praktisch unbegrenzt: 20 Jahre Monatsdaten (240 Perioden) laufen fehlerfrei.

18. Fallstricke

Die Fehler, die beim Bau tatsächlich aufgetreten sind. Sie sparen Zeit:

isFinite(null) ist true. Koordinaten dürfen `null` sein. Die Prüfung `isFinite(dc.lat)` lässt `null` durch, `+null` ergibt 0 — Standorte landen still im Golf von Guinea, Distanzen werden absurd. Es braucht eine strikte Prüfung `typeof v === 'number' && isFinite(v)`, konsequent an jeder Stelle.

Tausendertrennung mit führender Null. Die Regel „Ziffern, Trennzeichen, genau drei Ziffern“ trifft auch auf `0,042` zu und macht daraus 42. Die erste Ziffergruppe darf keine führende Null haben.

Chart.js: callback: undefined überschreibt den Standard. Eine Achsoption nur setzen, wenn ein Wert vorhanden ist — sonst zeigt die Kategorieachse Indizes statt Beschriftungen.

Chart.js zeichnet nicht in unsichtbare Flächen. Ist der Container `display:none`, ist die Zeichenfläche 0 px breit und `toDataURL()` liefert ein leeres Bild. Für Aufnahmen den Bereich kurz einblenden.

SheetJS `xlsx.full.min.js` ist kein UTF-8. Die Datei enthält binäre Codepage-Tabellen. Sie lässt sich nicht in eine HTML-Datei einbetten; die `core`-Variante ist sauber, halb so groß und deckt alle Formate ab.

Spalten dürfen nur einmal vergeben werden. Bei feldweiser Zuordnung mit Teiltreffern belegt eine Überschrift wie „Transportkosten je km“ zwei Felder gleichzeitig.

L.LayerGroup kennt kein `bringToFront()`. Nicht nötig: Leaflet legt Marker ohnehin in eine höhere Ebene als Kacheln.

Mehrzeilige HTML-Texte als Übersetzungsschlüssel. Der Textknoten enthält Zeilenumbrüche und Einrückung aus dem Quelltext. Beim Erfassen die Leerräume normalisieren, sonst greift kein Wörterbucheintrag.

Statische Texte vor dem ersten Rendern erfassen. Später enthielte das Dokument Nutzerdaten, die dann versehentlich als Schlüssel gelten würden.

19. Empfohlene Baureihenfolge

Diese Reihenfolge hat sich bewährt, weil jeder Schritt für sich testbar ist:

1. **Datenmodell und Zustandsverwaltung** samt Ereignissen und localStorage
2. **Stammdatenverwaltung** (DCs) mit Tabelle und Modal — die erste sichtbare Funktion, prüft das Ereignismodell
3. **Import** mit Spalten-Mapping. Aufwändiger als erwartet; Zahlen- und Periodenerkennung früh mit Testfällen absichern
4. **Rechenkern** ohne Oberfläche, direkt gegen Testdaten geprüft
5. **Simulationsansicht** mit Empfehlung, Rangliste und Score-Herleitung
6. **Dashboard** mit Kennzahlen und Diagrammen
7. **Karte** inklusive Offline-Abstieg
8. **Szenarien**
9. **Export**
10. **Zweisprachigkeit** — spät, aber nicht zu spät: Wer sie von Anfang an einplant und alle Texte über die Render-Helfer führt, spart den größten Teil des Aufwands

Ein **Demodatensatz** zahlt sich ab Schritt 4 aus: Er macht jeden folgenden Schritt sofort prüfbar, ohne dass echte Daten vorliegen müssen.